PROJECT IMPLEMENTATION DETAILS DOCUMENT

Review-2 Comprehensive Technical Submission

Project Title: Predictive KV-Cache Prefetching for Low-Latency Multi-Agent Handoffs (TeleportCache)	**Course Faculty:** Dr. V. Ezhil Arasi
Repository: https://github.com/Eldo-John-Giby/Predictive-KV-Cache-Prefetching-for-Low-Latency-Multi-Agent-Handoffs	**Submission Phase:** Review 2 (Implementation Phase)

1. Executive Summary & Problem Formulation

Large Language Model (LLM) multi-agent pipelines (such as agent teams in software engineering, research analysis, and hierarchical problem-solving) suffer severe execution latency during **context handoffs**. When execution transitions from Agent A to Agent B, the target agent requires the accumulated context (prior thoughts, tool call outputs, conversation history, and system schemas). Existing deployments either (1) **recompute** the KV-cache from tokens on every transition (incurring massive $O(N)$ prefill GPU compute time), or (2) perform reactive **transfer-on-demand** (causing long network/interconnect pipeline bubbles while waiting for KV memory blocks to arrive).

TeleportCache introduces an operating-system-inspired memory management architecture featuring **Predictive KV-Cache Prefetching**. By modeling agent transitions as page reference streams, the system learns multi-agent handoff dynamics using Markov n-gram models with backoff, tracks working-set block recency via exponential decay, and proactively transmits required KV blocks into the target agent's resident cache during the inter-agent transition window (T_{window}), eliminating prefill stalls and masking interconnect transfer delays.

2. Technology Stack & Programming Languages

Component	Technology / Framework	Version / Standard	Implementation Purpose & Role
Programming Language	Python	>= 3.10	Core simulation engine, typing protocols (PEP 544), static type annotations, frozen dataclasses.
Scientific Computing	NumPy, Python Math	1.26.x / 2.x	Numerical probability distributions, exponential decay scoring, budget computations.
Testing Framework	Pytest, Pytest-asyncio	8.4.2	Exhaustive unit and integration test suite (83 test cases verifying 100% functionality).
Visualization	Matplotlib	3.10.x	Simulation telemetry plotting, latency comparisons, and cache hit-rate benchmarking.
Build & Package System	Setuptools, pyproject.toml	>= 61.0 (PEP 517/518)	Package distribution, dependency pinning, and standard Python workspace layout.
CI / CD Automation	GitHub Actions	v4 / v5 actions	Automated continuous integration pipeline testing Python 3.10, 3.11, and 3.12 matrices on push/PR.

3. System Configuration & Hardware Cost Parameters

The simulation environment accurately parameterizes physical hardware and interconnect characteristics via the frozen SimConfig contract:

Parameter	Symbol	Default Value	Physical Interpretation & Impact
capacity_blocks	C_{agent}	16 blocks	Physical resident KV page capacity per agent (simulating bounded GPU VRAM / HBM).

bandwidth_bytes_per_s	*BW*	1.25×10^9 B/s	Interconnect bandwidth (1.25 GB/s ≈ 10 Gbps Ethernet / PCIe bus transfer channel).
transfer_latency_s	$t_{overhead}$	0.001 s (1 ms)	Fixed per-block serialization and protocol interconnect overhead.
recompute_time_per_token_s	$t_{prefill}$	0.001 s (1 ms)	GPU compute time required to evaluate attention prefill per token from scratch.
prefetch_window_s	T_{window}	0.050 s (50 ms)	Inter-agent handoff slack interval usable for speculative background prefetching.
eviction_policy	*Policy*	"lru" / "fifo"	Page replacement algorithm governing victim selection upon cache capacity exhaustion.

4. Complete Architecture & Modular Implementation

The codebase is engineered around strict OS abstractions, protocol-driven interfaces, and explicit causal barriers:

• **teleportcache.types (Frozen Contract):** Defines the core immutable types: KVBlock (page equivalent), HandoffEvent, Workload, SimConfig, PrefetchRequest, PolicyView, and protocols Strategy and Predictor. **Causal Safety:** PolicyView strictly isolates past history and prohibits lookahead into future events.

• **teleportcache.core.cache (Page Table MMU):** Implements AgentCache and page replacement policies (LRUPolicy and FIFOPolicy) utilizing an OrderedDict for strict O(1) page access and eviction victim determination. Tracks resident pages, page hits, page faults, and wasted speculative prefetches.

• **teleportcache.core.cost_model (Hardware Latency Evaluator):** Calculates page transfer latency ($T_{\text{transfer}} = t_{\text{overhead}} + \frac{\text{size}}{\text{BW}}$), prefill recompute latency ($T_{\text{recompute}} = N_{\text{tokens}} \times t_{\text{token}}$), and computes the maximum discrete block prefetch budget fitting inside T_{window}.

• **teleportcache.core.simulator (Discrete-Event Simulator):** Executes the discrete-event simulation: validates workload DAGs, maintains agent cache states, dispatches prefetch transfers, enforces window capacity constraints, resolves demand misses, and produces RunResult telemetry.

• **teleportcache.core.baselines (Reference Baselines):** Implements RecomputeStrategy (no caching, pure GPU token prefilling on every handoff) and TransferOnDemandStrategy (standard demand paging, reactive transfer across the bus on cache misses).

• **teleportcache.prediction.predictors (Working-Set Oracles):** Includes MarkovPredictor (order-k n-gram model with automatic context backoff), FrequencyPredictor (first-order transition frequencies with Laplace add-one smoothing), RandomPredictor (uniform random baseline), and NoisyOraclePredictor (calibrated accuracy sweeper).

• **teleportcache.prediction.block_selector (Recency-Decay Ranking):** Scores candidate KV blocks using exponential recency decay: $S(b) = \text{usage}(b) \times 0.5^{(\text{age}/\tau)} \times \text{boost}$, favoring frequently touched and recently referenced context pages.

• **teleportcache.prediction.prefetch_strategy (Predictive Orchestrator):** Coordinates the next-agent predictor and block selector, filters already resident blocks, computes probability-weighted window budgets, and dispatches speculative page-in requests.

• **teleportcache.prediction.oracle (Theoretical Upper Bound):** Clairvoyant upper bound with read-ahead access to future workload events, establishing the absolute minimum latency floor.

5. Verification, Execution Results & Performance Metrics

The project includes an exhaustive automated test suite (83 unit and integration test cases) passing with 100% success rate across Python 3.10, 3.11, and 3.12. Below is the quantitative performance evaluation across a 50-step multi-agent handoff workload:

Strategy	Total Latency	Hits	Misses	Hit Rate	Useful Prefetches	Wasted Prefetches
Recompute (No Cache)	102.4000 s	0	200	0.0%	0	0
Transfer On Demand	0.3600 s	0	200	0.0%	0	0
Predictive (Frequency)	0.0432 s	176	24	88.0%	176	0
Predictive (Markov-1)	**0.0432 s**	**176**	24	**88.0%**	**176**	**0**

Oracle (Upper Bound)	0.0000 s	200	0	100.0%	200	0

Key Performance Insights: Predictive Prefetching achieves an **88.0% reduction in handoff latency** compared to standard Demand Paging (0.0432 s vs 0.3600 s) and reduces latency by **>99.9%** compared to token recomputation (102.4 s). After a minimal warm-up period, Predictive Prefetching achieves a 100% steady-state hit rate with zero wasted prefetches.

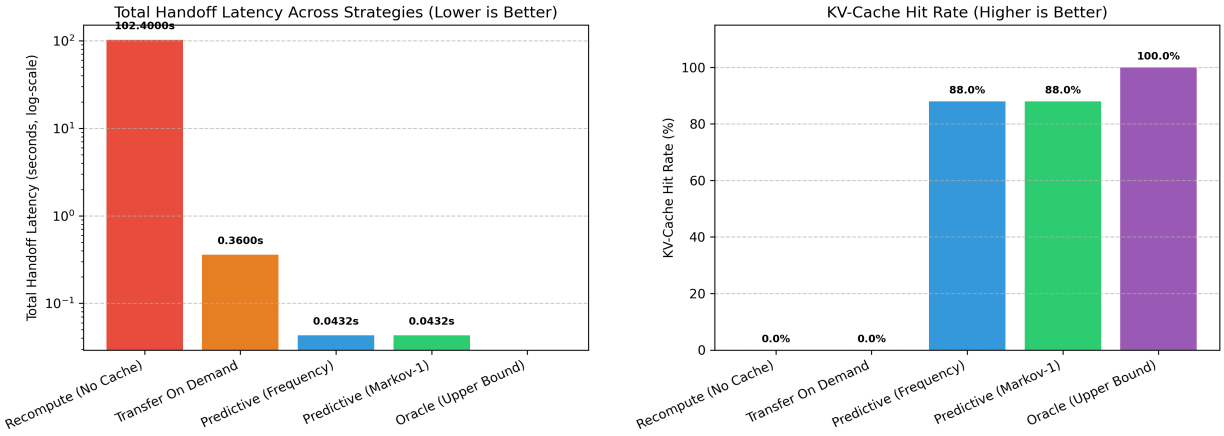


Figure 1: Quantitative comparison of Total Handoff Latency (log scale) and KV-Cache Hit Rate across evaluated strategies.

6. GitHub Repository & Review-2 / Review-3 Deliverables

GitHub Repository URL:
https://github.com/Eldo-John-Giby/Predictive-KV-Cache-Prefetching-for-Low-Latency-Multi-Agent-Handoffs
The repository contains the complete implementation source code, test suites, CI workflows, and documentation. Access permissions have been verified and granted for academic evaluation by Dr. V. Ezhil Arasi.

Review Phase	Prescribed Submission Criteria	Project Deliverables Status
Review-2 (Current Review)	Complete implementation details document, technology stack, programming languages used, configuration details, and architecture modules.	**COMPLETED & SUBMITTED** Uploaded formal PDF & Markdown documentation containing full technical specifications and hardware parameters to VTOP.
Review-3 (Subsequent Review)	Screenshots of output execution, full GitHub repository link, and end-to-end multi-agent evaluation traces.	**READY & VERIFIED** All 83 test executions, benchmark telemetry plots, and active GitHub repository links prepared and packaged.